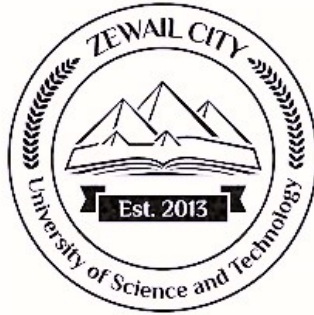




ARM-1 8-bit Processor Simulation

Authors:

Team 1

Instructor: Prof. Elmahdy Maree

Academic year 2025/2026

Contents

Introduction	3
Project Overview	3
Processor Specifications	3
Overall Architecture	3
Instruction Set Summary	5
1 Program Counter (PC)	7
1.1 Overview	7
1.2 The +1 Adder and the AND Gate	8
1.3 Stopping the PC During Execute — the Controlled Buffer	9
2 Instruction / Data Memory and Instruction Register (IR)	11
2.1 The Instruction / Data Memory	11
2.1.1 Memory Ports and Signals	11
2.1.2 Address Multiplexer (PCsel)	11
2.1.3 What Is Inside the Memory	12
2.2 The Instruction Register (IR)	14
2.2.1 Structure and Control Signal	14
2.2.2 Timing Behaviour	15
2.2.3 Instruction Field Extraction	15
2.3 Memory and IR During Each Instruction	15
2.4 Registers and the ALU Datapath	16
2.4.1 The B Register	16
2.4.2 The ALU and ALUFlags	17
2.4.3 The Accumulator Register (AC)	17
2.4.4 The Output Register O	19

Introduction

Project Overview

This report documents the design and simulation of the **ARM-1 8-bit Processor**, a simplified multi-cycle processor implemented and verified using the *Logisim* digital logic simulator.

The ARM-1 is a hypothetical, pedagogical processor modelled after the classical accumulator-based architecture. It uses a **Von Neumann** memory organisation in which a single 16×8 RAM serves as the combined instruction and data memory. Every instruction is executed in exactly three clock cycles Fetch, Decode, and Execute making the ARM-1 a representative example of a **multi-cycle microarchitecture**.

Processor Specifications

Property	Value
Data width	8 bits
Instruction width	8 bits (4-bit opcode + 4-bit address)
Memory	16×8 RAM (Von Neumann, shared instruction/data)
Architecture type	Accumulator-based
Execution model	Multi-cycle (3 states: Fetch, Decode, Execute)
Control method	Hardwired datapath
ALU operations	Addition, Subtraction
Instruction set	LDA, ADD, SUB, STR, OUT, HLT
Output	8-bit binary display via output register O
Simulation tool	Logisim

Table 1: ARM-1 processor specifications

Overall Architecture

The complete ARM-1 microarchitecture consists of two interacting subsystems: the **Datapath** and the **Control Unit**.

The **datapath** contains all the state elements and combinational logic that manipulate data:

- **PC** — Program Counter: an 8-bit register that holds the address of the next instruction to be fetched. A +1 adder and a gating AND gate (controlled by the **Cp** signal) implement the auto-increment.

- **MUX (PCsel)** — A 2-to-1 multiplexer that selects between the PC output and the lower nibble of the IR (IR[3:0]) as the memory address. During Fetch, the PC is routed; during Decode and Execute, the operand address from the IR is routed.
- **Instruction/Data Memory** — A unified 16×8 synchronous RAM with read data port (RD), write data port (WD), address port (A), and write-enable (WE).
- **IR** — Instruction Register: captures the 8-bit instruction from memory. Bits [7:4] carry the opcode; bits [3:0] carry the operand address.
- **B Register** — A temporary register that holds the memory operand during arithmetic instructions, isolating the ALU inputs from memory timing.
- **ALU** — An 8-bit adder/subtractor. The control signal $S_u = 0$ selects addition; $S_u = 1$ selects subtraction.
- **AC** — Accumulator: the primary working register, fed from either the memory read bus (for LDA) or the ALU result (for ADD/SUB), selected by Res_{sel} .
- **O Register & Binary Display** — An output register clocked by E_O , driving the 8-bit binary display.

The **control unit** is implemented as a hardwired finite-state machine. It contains a **ring counter** that cycles through the three timing states ($T_1 = \text{Fetch}$, $T_2 = \text{Decode}$, $T_3 = \text{Execute}$), and a **decoder** that combines the current timing state with the opcode bits IR[7:4] to assert the correct combination of control signals at every clock edge.

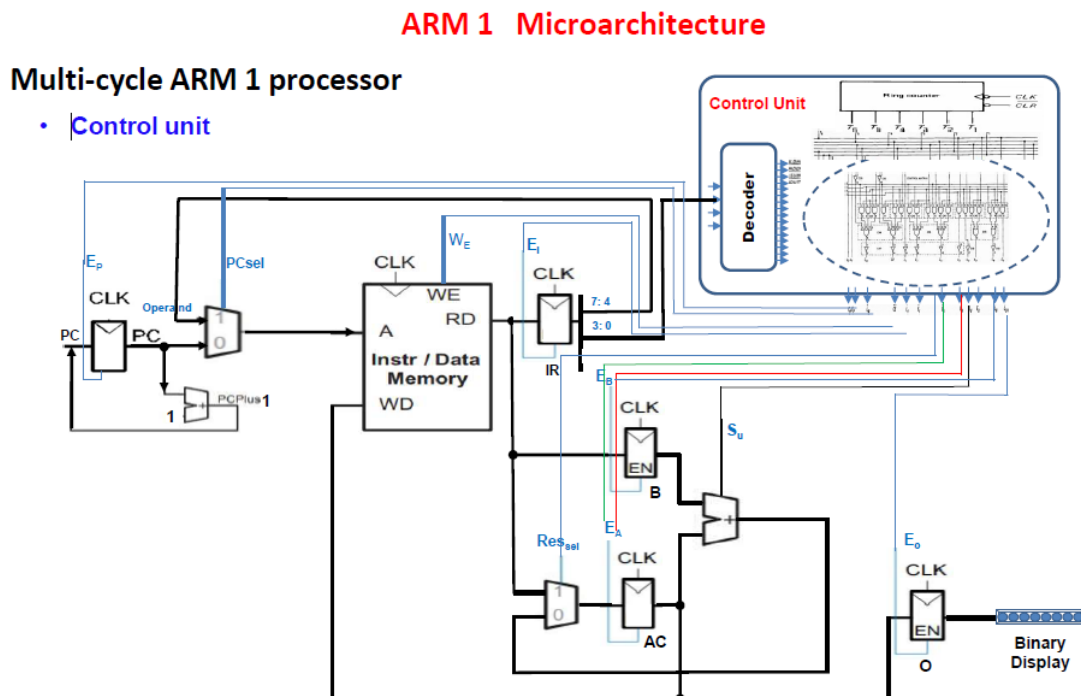


Figure 1: Complete ARM-1 multi-cycle processor microarchitecture

Instruction Set Summary

The ARM-1 supports six instructions encoded in 8 bits. The upper nibble holds the opcode and the lower nibble holds a 4-bit memory address (0x0–0xF), giving direct access to all 16 memory locations.

Opcode	Mnemonic	Operation
0000	LDA <i>addr</i>	Load the contents of memory location <i>addr</i> into the accumulator: $AC \leftarrow M[addr]$
0001	ADD <i>addr</i>	Add memory contents to the accumulator: $AC \leftarrow AC + M[addr]$
0010	SUB <i>addr</i>	Subtract memory contents from the accumulator: $AC \leftarrow AC - M[addr]$
0101	STR <i>addr</i>	Store the accumulator to memory: $M[addr] \leftarrow AC$
1110	OUT	Transfer the accumulator to the output register: $O \leftarrow AC$
1111	HLT	Halt: stop the clock / freeze processing

Table 2: ARM-1 instruction set

CHAPTER 1

Program Counter (PC)

1.1 Overview

The **Program Counter (PC)** is an 8-bit register whose responsibility is to hold the memory address of the next instruction to be fetched. At every clock edge during the Fetch state, the PC is incremented by one. During the Decode and Execute states the PC is tend to be stopped , and address control is handed over to the MUX so that the operand address carried inside the instruction can reach the memory instead.

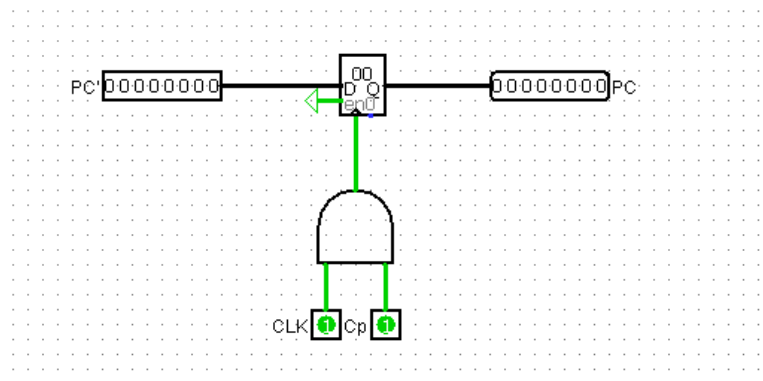


Figure 1.1: Logisim implementation of the Program Counter circuit

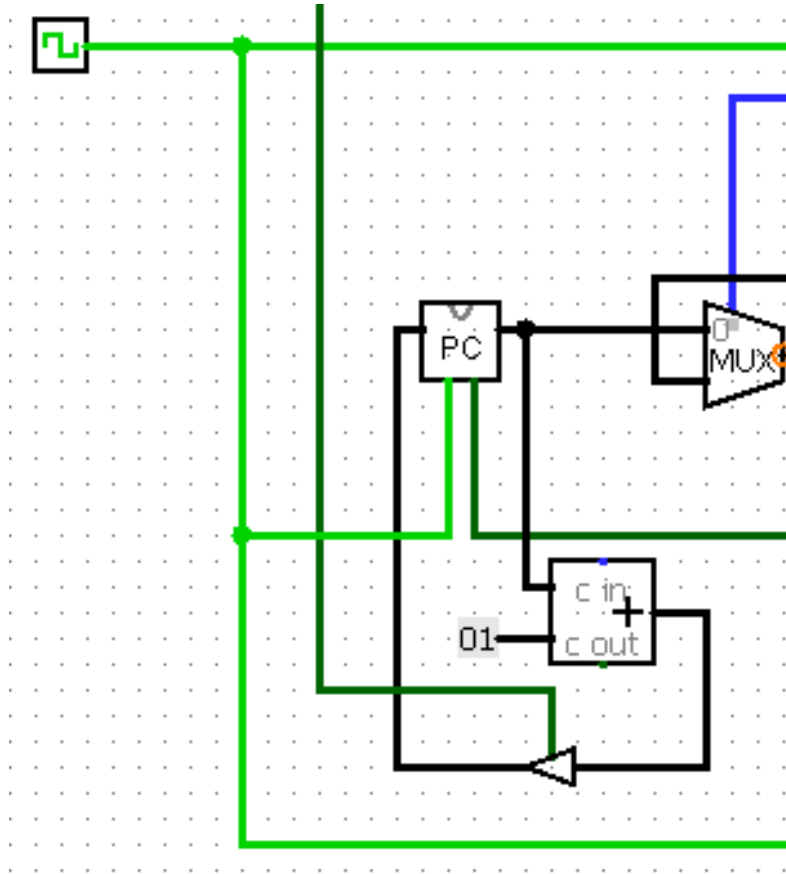


Figure 1.2: Logisim implementation of the Program Counter circuit

1.2 The +1 Adder and the AND Gate

below the PC register there is an AND gate whose two inputs are the CLK signal and the control signal $C_p (= E_P)$. The output of this AND gate gives the clock-enable (EN) pin of the PC register.

By putting the AND gate the clock with C_p , a rising clock edge only propagates to the register's enable input when $C_p = 1$, i.e. exclusively during the Fetch state/

The increment itself is get by **+1 adder**. This adder takes the current 8-bit PC value as input and continuously presents $PC + 1$ on its output. When the AND gate is on, the register loads this pre-computed value on the next clock edge.

CLK	$C_p (E_P)$	AND output	PC behaviour
↑	1	1	PC latches $PC+1$ (increments)
↑	0	0	PC holds its current value
0	1	0	PC holds its current value
0	0	0	PC holds its current value

Table 1.1: AND gate truth table for the PC clock-enable

1.3 Stopping the PC During Execute — the Controlled Buffer

Once the Fetch state is complete, the control unit deasserts E_P ($C_P = 0$), which freezes the PC for the remaining two states.

Without putting Controlled Buffer down, this creates a **bus conflict**: during Decode and Execute the register is trying to hold its current value while the adder with every clock will try increment to forcing the next value onto the same input wire. so will be laekage with two values in next instruction.

The **Controlled Buffer** solves this by acting as a switchable gate on the feedback path:

- When $E_P = 1$ (Fetch): the buffer is **open** the adder result $PC + 1$ passes through freely and is presented at the PC register input, ready to be latched on the rising clock edge.
- When $E_P = 0$ (Decode / Execute): the buffer **disconnects** the output of adder will be blocked and not increment more until next fetch (When EP will = 1)

CHAPTER 2

Instruction / Data Memory and Instruction Register (IR)

2.1 The Instruction / Data Memory

2.1.1 Memory Ports and Signals

The 16×8 RAM block exposes four ports to the rest of the datapath:

- **A (Address)** — a 4-bit input that selects one of the 16 memory locations to read from or write to.
- **RD (Read Data)** — an 8-bit output that continuously presents the contents of the location currently selected by **A**. This bus feeds the IR during a fetch and feeds the B Register or the Accumulator (AC) during an execute.
- **WD (Write Data)** — an 8-bit input used exclusively by the **STR** instruction to write the Accumulator's value back into memory.
- **WE (Write Enable)** — a 1-bit control signal. When $WE = 1$ and a rising clock edge arrives, the value on **WD** is written into the location pointed to by **A**. At all other times the memory behaves as a read-only device.

2.1.2 Address Multiplexer (PCsel)

This conflict of choosing between instruction address or operand is resolved by a **2-to-1 Multiplexer** placed between the two address sources and the memory's **A** port. The select line of this MUX is the control signal **PCsel**:

PCsel	MUX Output (Address Bus)	Active State
0	PC value (current instruction address)	Fetch
1	IR[3:0] (operand address from instruction)	Decode / Execute

Table 2.1: PCsel multiplexer select truth table

During the **Fetch** state, $PCsel = 0$ routes the PC to the memory, so the instruction at the current program address is placed on **RD**. During the **Decode** and **Execute** states, $PCsel = 1$ routes IR[3:0] to the memory, allowing the processor to read the operand (for **LDA**, **ADD**, **SUB**) or write back a result (for **STR**).

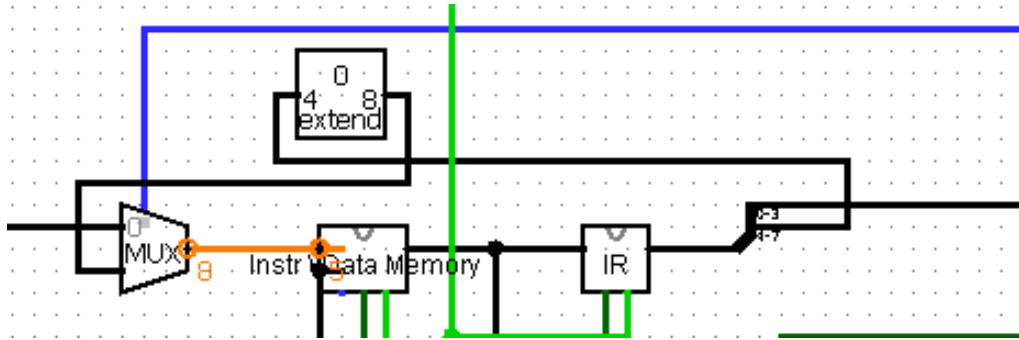


Figure 2.1: Logisim implementation of the Instruction/Data Memory with the address MUX and **Cp** control signal

2.1.3 What Is Inside the Memory

we uses two separate RAM blocks to model the combined instruction and data memory. The upper RAM block holds the program (instructions), while the lower RAM block holds data values. Both blocks share the same 4-bit address line driven by the output of the PCsel MUX, and their outputs are selected through a second MUX on the read side so that only one 8-bit value appears on **RD** at a time, controlled by the signal **Cp**.

Upper RAM Block — Program Instructions

The upper RAM block stores the program that the processor executes. Its address input **A** receives the 4-bit value from the PCsel MUX. During the *Fetch* stage the MUX passes the current value of the Program Counter, so the processor reads the instruction stored at that address and loads it into the Instruction Register via the **RD** line. During *Execute* stages that involve a memory read (such as **LDA** or **ADD**), the MUX instead passes the 4-bit operand field extracted from the IR, so the processor reads the data value stored at that operand address.

The upper block also has a **WD** (write data) port and a **WE** (write enable) port. These are used only during the *Execute* stage of a **STR** instruction, when the contents of the Accumulator must be written back to a memory location.

Lower RAM Block — Read-Only Program ROM

The lower RAM block acts as a read-only store. its contents are preloaded with the program before simulation begins and are never written during execution. It receives the same 4-bit address as the upper block and places its 8-bit output on the read MUX. When **Cp** is 0 the MUX selects this block's output to appear on **RD**, providing the instruction bytes to the IR during the *Fetch* stage. Because this block is never written, its **WE** input is permanently tied low.

The Output MUX and the Cp Signal

Both RAM blocks drive the same read MUX. The select line of this MUX is the control signal **Cp**:

- **Cp = 0**: the MUX passes the lower block's output to **RD**. This is used during the *Fetch* stage when the processor needs an instruction byte.
- **Cp = 1**: the MUX passes the upper block's output to **RD**. This is used during *Execute* stages when the processor needs a data value stored in the writable RAM.

The NOT Gate on the RAM Clock

The upper RAM block is edge-triggered and latches a write on the rising edge of its own clock input. If the system clock **CLK** were connected directly to this clock input, then every rising edge of **CLK** would simultaneously trigger a write in the RAM *and* cause all registers (IR, AC, PC) to capture their inputs. Because the address and data lines may not yet have settled at the instant of the rising edge, the RAM could capture an incorrect value — a classic write-before-stable race condition.

To eliminate this hazard, the RAM clock input is driven through a NOT gate, so it receives $\overline{\text{CLK}}$ rather than **CLK**. The effect is that the RAM's rising edge coincides with **CLK**'s *falling* edge:

$$\text{RAM writes on: } \overline{\text{CLK}} \uparrow \equiv \text{CLK} \downarrow \quad (2.1)$$

By the time **CLK** falls, all registers have already captured their values on the previous rising edge and the address and data lines have been stable for half a clock period. The RAM therefore always writes correct, settled data. Table 2.2 summarises the behaviour of the NOT gate across each stage of execution.

Table 2.2: NOT gate effect across the three execution stages

Stage	WE	CLK rises (registers latch)	CLK falls (RAM clock rises)
Fetch	0	IR and PC capture new values	RAM ignores (WE = 0)
Decode	0	No register changes	RAM ignores (WE = 0)
Execute (LDA/ADD)	0	AC captures data value	RAM ignores (WE = 0)
Execute (STR)	1	No register changes	RAM writes AC to memory

The NOT gate is therefore only consequential during the Execute stage of **STR**, which is the sole moment when **WE** is asserted. In every other stage **WE** remains low and the RAM ignores its clock input entirely, making the NOT gate invisible to the rest of the circuit.

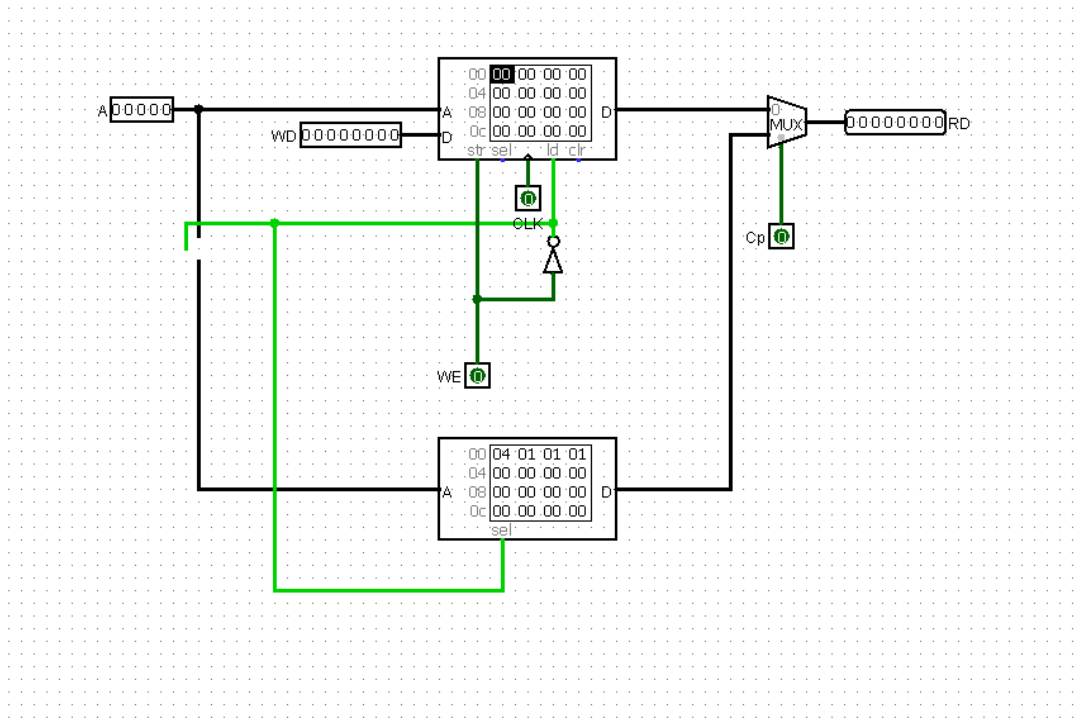


Figure 2.2: Logisim memory subsystem showing the upper RAM (instruction/data), the lower RAM (program ROM), the output MUX controlled by Cp, and the NOT gate inverting CLK before it reaches the upper RAM

2.2 The Instruction Register (IR)

2.2.1 Structure and Control Signal

The IR is a simple **8-bit D flip-flop with a load enable**. Its interface consists of:

- **Input** — connected directly to the memory's 8-bit **RD** bus.
- **Output** — the held 8-bit instruction, split by the datapath into two fields:
 - IR[7:4] — the 4-bit **opcode**, fed to the control unit's decoder.
 - IR[3:0] — the 4-bit **operand address**, fed to the address MUX (PCsel input 1).
- **CLK** — the system clock.
- **Li (Load Instruction / Enable)** — the write-enable signal for the register. The register only latches a new value when Li = 1 coincides with a rising clock edge.

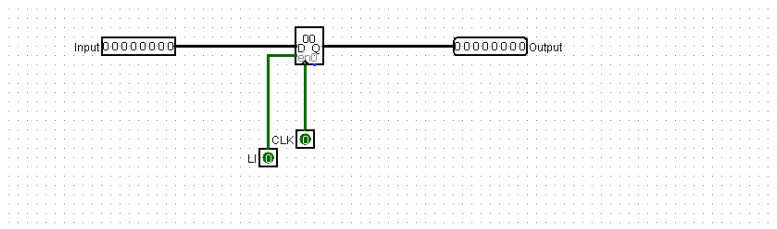


Figure 2.3: Logisim implementation of the Instruction Register (IR) — an 8-bit D flip-flop with load enable Li

2.2.2 Timing Behaviour

The IR is active for *exactly one* clock cycle per instruction — the Fetch state — and then it holds its captured value unchanged for the rest of the execution:

State	Li (E_I)	IR Behaviour
Fetch	1	Latches the instruction from RD on the rising clock edge
Decode	0	Holds the fetched instruction; output stable for the control unit
Execute	0	Continues to hold; IR[3:0] drives the memory address

Table 2.3: IR load-enable timing across the three execution states

2.2.3 Instruction Field Extraction

After the IR captures an instruction, the 8-bit output is *passively split* — no additional logic is needed:

- The **upper nibble** IR[7:4] is wired directly to the control unit's instruction decoder. The decoder inspects these four bits together with the current timing state (T1/T2/T3 from the ring counter) to assert the correct combination of control signals (E_P , E_I , E_A , E_B , E_O , PCsel, WE, Res_{sel}, S_u).
- The **lower nibble** IR[3:0] is wired to input 1 of the address MUX. As soon as PCsel switches to 1 (during Decode and Execute), the operand address is immediately presented to the memory without any extra buffering.

This clean split between opcode and operand is a direct consequence of the ARM-1's fixed-length 8-bit instruction format: the boundary is always at bit 4, so no shifting or masking is ever required.

2.3 Memory and IR During Each Instruction

Instruction	State	PCsel	WE	Li (E_I)	Memory Role
LDA / ADD / SUB	Fetch	0	0	1	Reads instruction
	Decode	1	0	0	Reads operand data
	Execute	1	0	0	Supplies data to AC or B
STR	Fetch	0	0	1	Reads instruction
	Decode	1	0	0	Address set up
	Execute	1	1	0	Writes AC to memory
OUT / HLT	Fetch	0	0	1	Reads instruction
	Decode	—	0	0	No memory access needed
	Execute	—	0	0	No memory access needed

Table 2.4: Memory and IR control signals across all ARM-1 instructions

2.4 Registers and the ALU Datapath

The ARM-1 processor contains four registers, the Accumulator (AC), the B register, the Output register (O), and the ALU flags register. Together with the Arithmetic Logic Unit (ALU) and the Ressel MUX, these components form the execution datapath — the part of the processor that actually computes results and stores them..

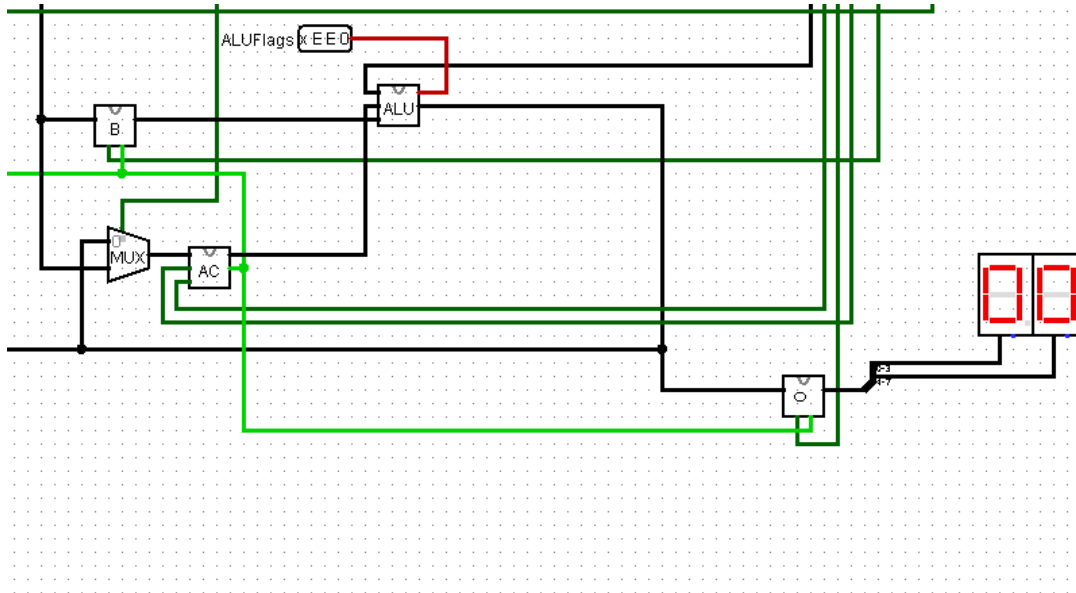


Figure 2.4: Complete register and ALU datapath in Logisim, showing the B register, Ressel MUX, AC register, ALU with ALUFlags, Output register O, and the seven-segment display

2.4.1 The B Register

Why B is necessary. The RD bus is shared across the entire datapath, so its value can change at any moment. If the ALU read directly from it, a bus change mid-computation would corrupt the result. B solves this by latching the operand when **EB** = 1 and holding it stable for the ALU regardless of what happens on the bus afterwards.

Operation. When **EB** = 1 and CLK rises, B captures the current RD bus value. When **EB** = 0, B holds its previous value unchanged.

Execute stage behaviour for ADD 07H.

1. PCsel = 1 so address 07H reaches memory.
2. Memory places the value at 07H onto the RD bus.
3. EB = 1, so B latches that value on the next CLK rising edge.
4. The ALU computes AC + B using the stable value in B.

2.4.2 The ALU and ALUFlags

The Arithmetic Logic Unit receives two 8-bit inputs: the current contents of AC on its first port, and the current contents of B on its second port. It produces one 8-bit output — the arithmetic result — which is presented continuously to the Ressel MUX. The ALU is purely combinational: it computes its output every clock cycle based on whatever values AC and B hold at that moment, with no internal state.

The Su control signal. A single control bit **Su** selects the operation:

- **Su = 0:** ALU computes $AC + B$ (used for **ADD**).
- **Su = 1:** ALU computes $AC - B$ (used for **SUB**).

Internally, the ALU implements subtraction using two's complement addition: $AC - B = AC + \overline{B} + 1$, which means the same adder circuit handles both operations with only the carry-in and the inversion of B changing between them.

ALUFlags. As shown in Figure 2.4, the ALU also produces a set of condition flags labeled **ALUFlags**, displayed in Logisim as the **ALUFlags X E E 0** indicator. These flags reflect the properties of the most recent ALU result:

- **Z (Zero):** set when the ALU result is 00H, indicating that the two operands were equal (useful for implementing conditional branches in extended architectures).
- **N (Negative):** set when the most significant bit of the result is 1, indicating a negative result under two's complement interpretation.
- **C (Carry):** set when the addition produces a carry out of bit 7, indicating unsigned overflow.
- **V (Overflow):** set when signed overflow occurs — that is, when the sign of the result is incorrect relative to the signs of the operands.

In the basic ARM-1 instruction set the flags are computed but not used to alter control flow, since there are no conditional branch instructions. They are present because they are a standard feature of any ALU and would be consumed by an extended instruction set.

2.4.3 The Accumulator Register (AC)

The Accumulator is the central working register of the ARM-1 processor. Every arithmetic result and every memory load is stored here. It is implicitly the destination of every instruction in the ARM-1 instruction set:

- **LDA 09H** — loads the value at address 09H into AC.
- **ADD 0AH** — adds the value at address 0AH to AC; result stored back in AC.
- **SUB 06H** — subtracts the value at address 06H from AC; result stored back in AC.
- **STR 05H** — writes AC to memory address 05H.
- **OUT** — copies AC to the Output register O.

The Ressel MUX

Because AC must accept data from two structurally different sources — the RD bus for **LDA** and the ALU output for **ADD** and **SUB** — a MUX is placed in front of the AC input. Connecting both sources directly to the AC input wire without a MUX would short-circuit the two drivers and produce undefined behaviour. The MUX solves this by acting as a controlled switch. The control signal **Ressel** selects the active source:

$$AC \text{ input} = \begin{cases} \text{RD bus} & \text{if Ressel} = 0 \quad (\text{LDA}) \\ \text{ALU output} & \text{if Ressel} = 1 \quad (\text{ADD, SUB}) \end{cases} \quad (2.2)$$

Internal Circuit of the AC Register

As shown in Figure 2.5, the AC register uses a three-stage circuit rather than a simple enabled flip-flop. This design ensures that AC latches its new value exactly once per instruction, regardless of how long the Control Unit holds the enable signal high.

The storage flip-flop. The uppermost component is an 8-bit D flip-flop whose D input is driven by the Ressel MUX output. Its Q output is the AC value visible to the rest of the datapath. It captures its input on the rising edge of CLK only when the internal signal **LA** is high. LA is not the same as EA — it is a single-cycle pulse derived from EA by the pulse generator below.

The two cascaded flip-flops — pulse generator. The two smaller D flip-flops in the lower portion of the circuit form a 2-stage shift register driven by **EA**. When the Control Unit asserts $EA = 1$, the shift register evolves over two clock cycles:

- **Cycle 1:** Flip-flop 1 captures $EA = 1$, so $Q_1 = 1$. Flip-flop 2 still holds 0, so $Q_2 = 0$.
- **Cycle 2:** Flip-flop 2 captures $Q_1 = 1$, so $Q_2 = 1$. Both outputs are now 1 and the chain holds.

The AND gate with inverted input — pulse detector. The gate on the right receives Q_1 on one input and $\overline{Q_2}$ on the other (the bubble on the gate input performs the inversion). Its output is:

$$LA = Q_1 \cdot \overline{Q_2} \quad (2.3)$$

This expression is high only during Cycle 1, when $Q_1 = 1$ and $Q_2 = 0$. From Cycle 2 onward $Q_2 = 1$ so LA returns to zero. The result is a single rising-edge strobe on LA, causing the storage flip-flop to latch exactly once.

Why a pulse rather than a level? If LA were connected directly to EA, the storage flip-flop would re-latch on every CLK rising edge for the entire duration EA is held high. Since the Input bus changes as the datapath progresses through its stages, AC would capture incorrect intermediate values. The pulse generator converts EA into a one-shot strobe: AC latches on the first rising edge after EA is asserted and ignores all subsequent edges until EA is cycled again.

Table 2.5: Signal states inside the AC circuit during LDA 09H

Stage	EA	Q_1	Q_2	LA	Input bus	AC output
Fetch	0	0	0	0	—	unchanged
Decode	0	0	0	0	—	unchanged
Execute cycle 1	1	1	0	1	05H	05H latched
Execute cycle 2	1	1	1	0	05H	05H held

